

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import os
import warnings
```

Data Collection and Cleaning

```
ab = pd.read_csv(r"C:\Users\HP\OneDrive\Desktop\Churn_Modelling.csv")
```

```
ab.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10000 entries, 0 to 9999
```

```
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	RowNumber	10000 non-null	int64
1	CustomerId	10000 non-null	int64
2	Surname	10000 non-null	object
3	CreditScore	10000 non-null	int64
4	Geography	10000 non-null	object
5	Gender	10000 non-null	object
6	Age	10000 non-null	int64
7	Tenure	10000 non-null	int64
8	Balance	10000 non-null	float64
9	NumOfProducts	10000 non-null	int64
10	HasCrCard	10000 non-null	int64
11	IsActiveMember	10000 non-null	int64
12	EstimatedSalary	10000 non-null	float64
13	Exited	10000 non-null	int64

```
dtypes: float64(2), int64(9), object(3)
```

```
memory usage: 1.1+ MB
```

```
ab.describe()
```

	RowNumber	CustomerId	CreditScore	Age
Tenure \				
count	10000.000000	1.000000e+04	10000.000000	10000.000000
mean	5000.500000	1.569094e+07	650.528800	38.921800
std	2886.89568	7.193619e+04	96.653299	10.487806
min	1.000000	1.556570e+07	350.000000	18.000000
25%	2500.750000	1.562853e+07	584.000000	32.000000
50%	5000.500000	1.569074e+07	652.000000	37.000000

```

5.000000
75%      7500.25000  1.575323e+07  718.000000  44.000000
7.000000
max      10000.00000  1.581569e+07  850.000000  92.000000
10.000000

```

```

      Balance  NumOfProducts  HasCrCard  IsActiveMember \
count  10000.000000  10000.000000  10000.000000  10000.000000
mean    76485.889288    1.530200    0.70550    0.515100
std     62397.405202    0.581654    0.45584    0.499797
min       0.000000    1.000000    0.00000    0.000000
25%      0.000000    1.000000    0.00000    0.000000
50%     97198.540000    1.000000    1.00000    1.000000
75%    127644.240000    2.000000    1.00000    1.000000
max    250898.090000    4.000000    1.00000    1.000000

```

```

      EstimatedSalary  Exited
count  10000.000000  10000.000000
mean    100090.239881    0.203700
std     57510.492818    0.402769
min      11.580000    0.000000
25%     51002.110000    0.000000
50%    100193.915000    0.000000
75%    149388.247500    0.000000
max    199992.480000    1.000000

```

```
ab.isnull().sum()
```

```

RowNumber      0
CustomerId     0
Surname        0
CreditScore    0
Geography     0
Gender         0
Age           0
Tenure        0
Balance       0
NumOfProducts 0
HasCrCard     0
IsActiveMember 0
EstimatedSalary 0
Exited        0
dtype: int64

```

```
ab.head()
```

```

      RowNumber  CustomerId  Surname  CreditScore  Geography  Gender  Age
\
0            1    15634602  Hargrave           619    France  Female   42

```

1	2	15647311	Hill	608	Spain	Female	41
2	3	15619304	Onio	502	France	Female	42
3	4	15701354	Boni	699	France	Female	39
4	5	15737888	Mitchell	850	Spain	Female	43

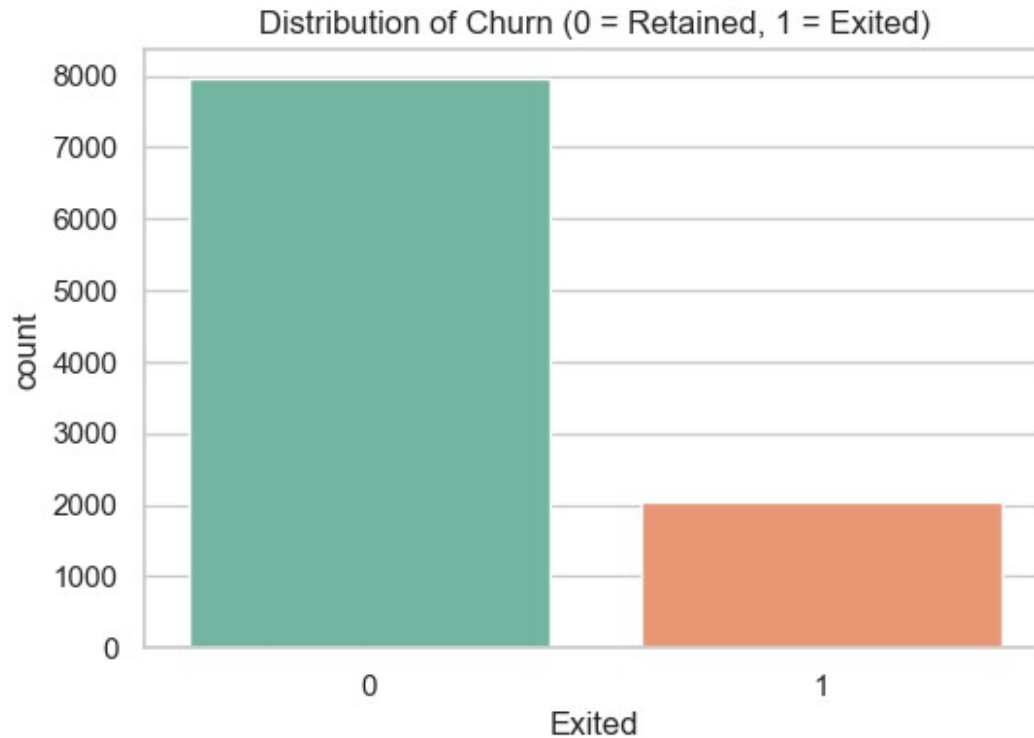
	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	
3	1	0.00	2	0	0	
4	2	125510.82	1	1	1	

	EstimatedSalary	Exited
0	101348.88	1
1	112542.58	0
2	113931.57	1
3	93826.63	0
4	79084.10	0

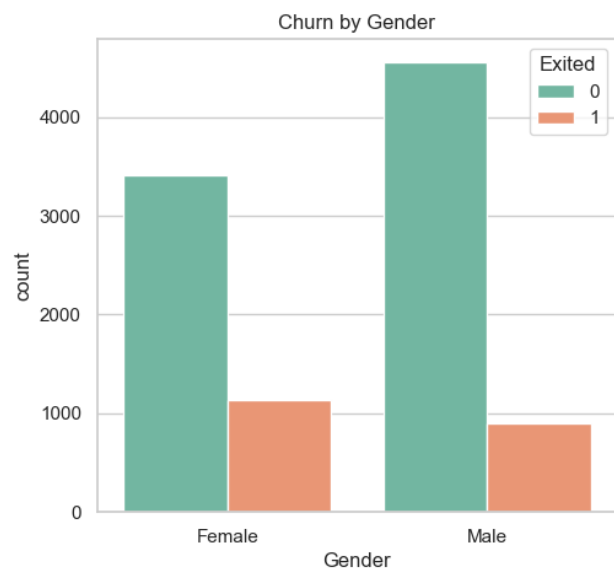
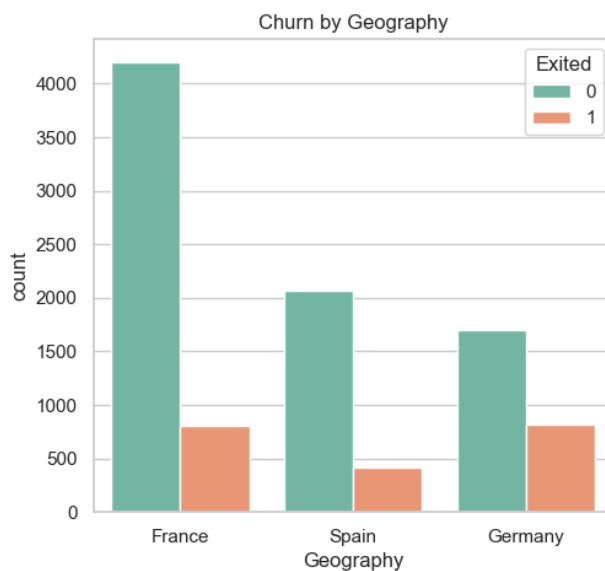
EDA & Visualisation

```
sns.set_theme(style="whitegrid")

plt.figure(figsize=(6, 4))
sns.countplot(data=ab, x='Exited', palette='Set2')
plt.title('Distribution of Churn (0 = Retained, 1 = Exited)')
plt.show()
```



```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sns.countplot(data=ab, x='Geography', hue='Exited', palette='Set2',
ax=axes[0])
axes[0].set_title('Churn by Geography')
sns.countplot(data=ab, x='Gender', hue='Exited', palette='Set2',
ax=axes[1])
axes[1].set_title('Churn by Gender')
plt.show()
```



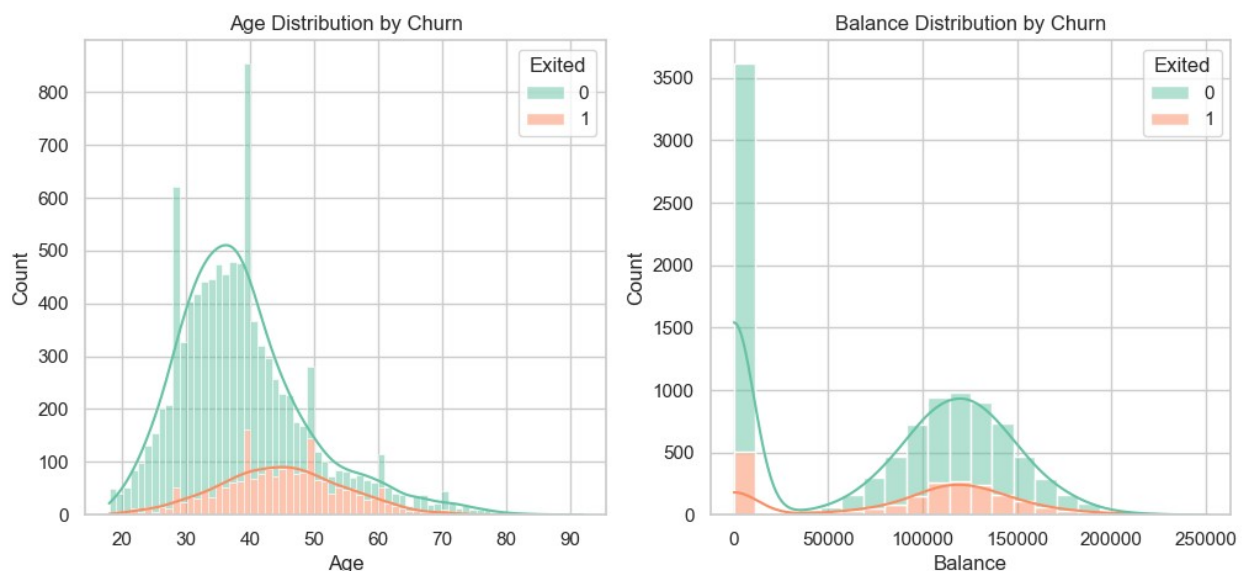
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sns.histplot(data=ab, x='Age', hue='Exited', multiple="stack",
palette='Set2', ax=axes[0], kde=True)
axes[0].set_title('Age Distribution by Churn')
sns.histplot(data=ab, x='Balance', hue='Exited', multiple="stack",
palette='Set2', ax=axes[1], kde=True)
axes[1].set_title('Balance Distribution by Churn')
plt.show()
```

D:\Users\HP\anaconda3\Lib\site-packages\seaborn_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.

```
with pd.option_context('mode.use_inf_as_na', True):
```

D:\Users\HP\anaconda3\Lib\site-packages\seaborn_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.

```
with pd.option_context('mode.use_inf_as_na', True):
```



```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

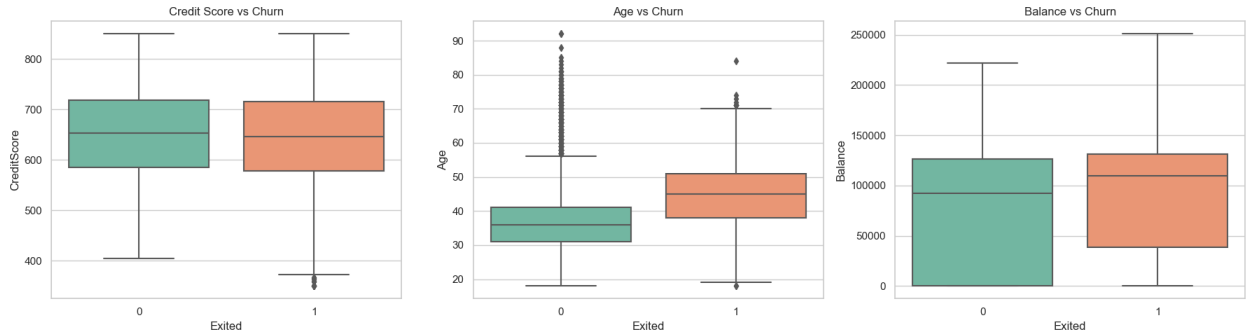
sns.boxplot(data=ab, x='Exited', y='CreditScore', palette='Set2',
ax=axes[0])
axes[0].set_title('Credit Score vs Churn')

sns.boxplot(data=ab, x='Exited', y='Age', palette='Set2', ax=axes[1])
axes[1].set_title('Age vs Churn')

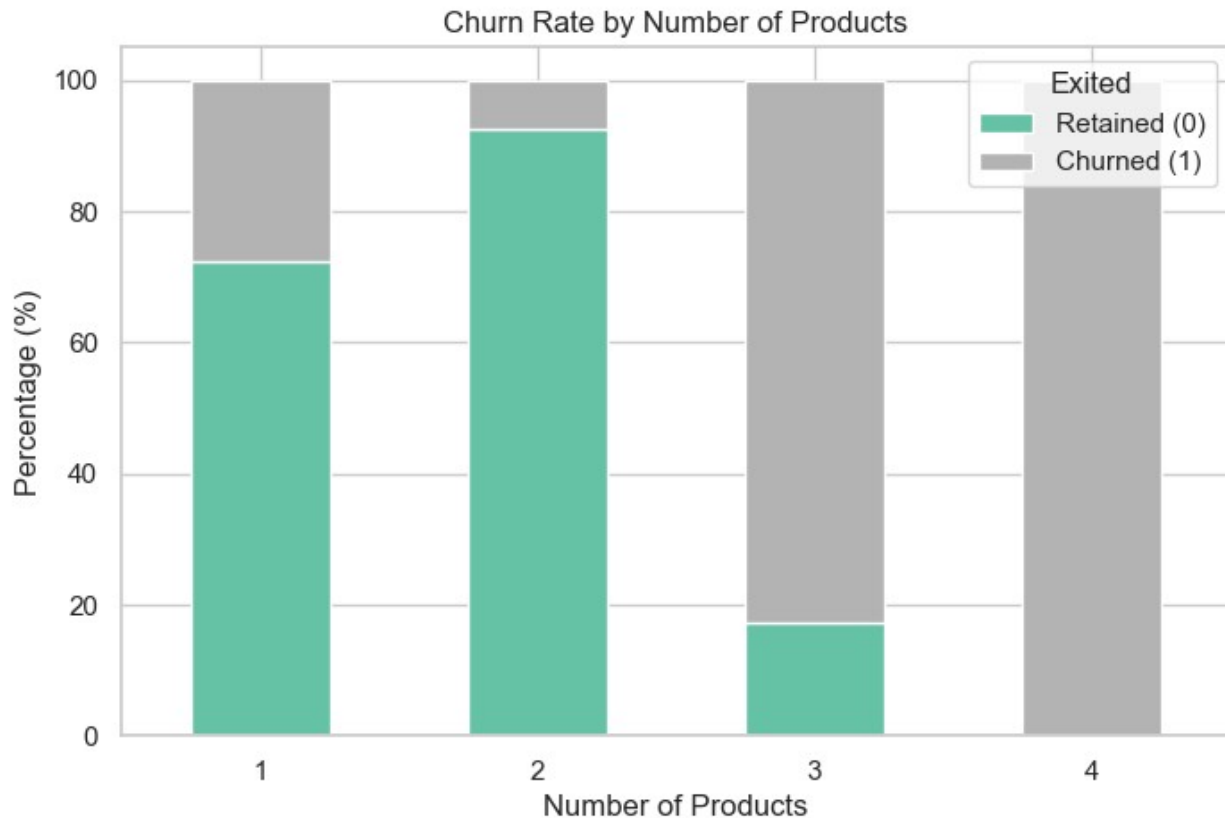
sns.boxplot(data=ab, x='Exited', y='Balance', palette='Set2',
ax=axes[2])
```

```
axes[2].set_title('Balance vs Churn')
```

```
plt.tight_layout()  
plt.show()
```



```
product_churn = pd.crosstab(ab['NumOfProducts'], ab['Exited'],  
                             normalize='index') * 100  
  
product_churn.plot(kind='bar', stacked=True, figsize=(8, 5),  
                    colormap='Set2')  
plt.title('Churn Rate by Number of Products')  
plt.ylabel('Percentage (%)')  
plt.xlabel('Number of Products')  
plt.legend(title='Exited', labels=['Retained (0)', 'Churned (1)'])  
plt.xticks(rotation=0)  
plt.show()
```



```
g = sns.FacetGrid(ab, col="IsActiveMember", hue="Exited", height=5,
aspect=1.2, palette='Set2')
g.map(sns.kdeplot, "Age", fill=True, alpha=0.5)
g.add_legend(title="Exited")
g.fig.subplots_adjust(top=0.85)
g.fig.suptitle('Age Density by Active Status and Churn')
plt.show()
```

```
D:\Users\HP\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
```

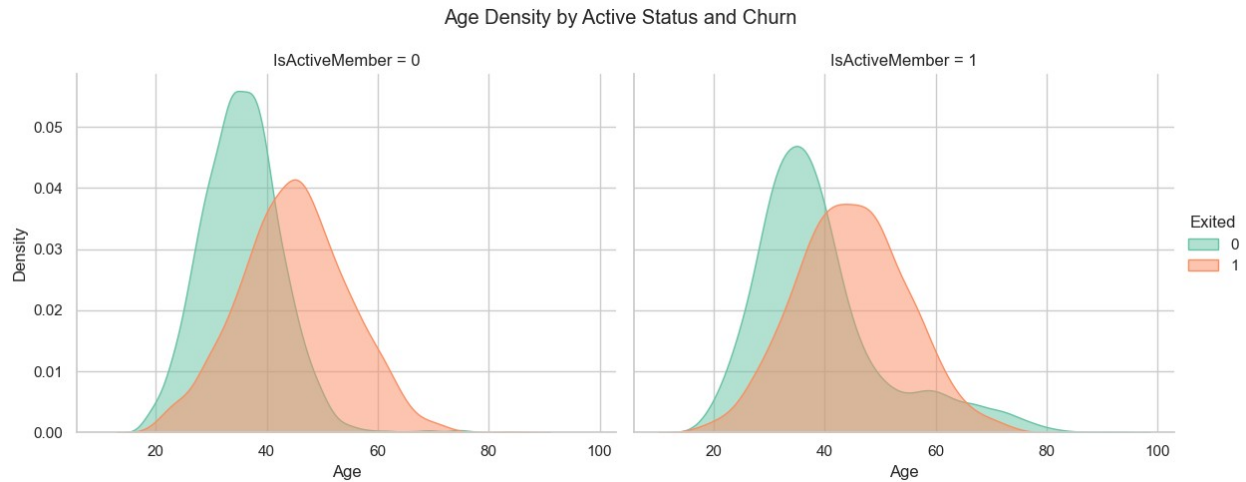
```
with pd.option_context('mode.use_inf_as_na', True):
D:\Users\HP\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
```

```
with pd.option_context('mode.use_inf_as_na', True):
D:\Users\HP\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed
in a future version. Convert inf values to NaN before operating
instead.
```

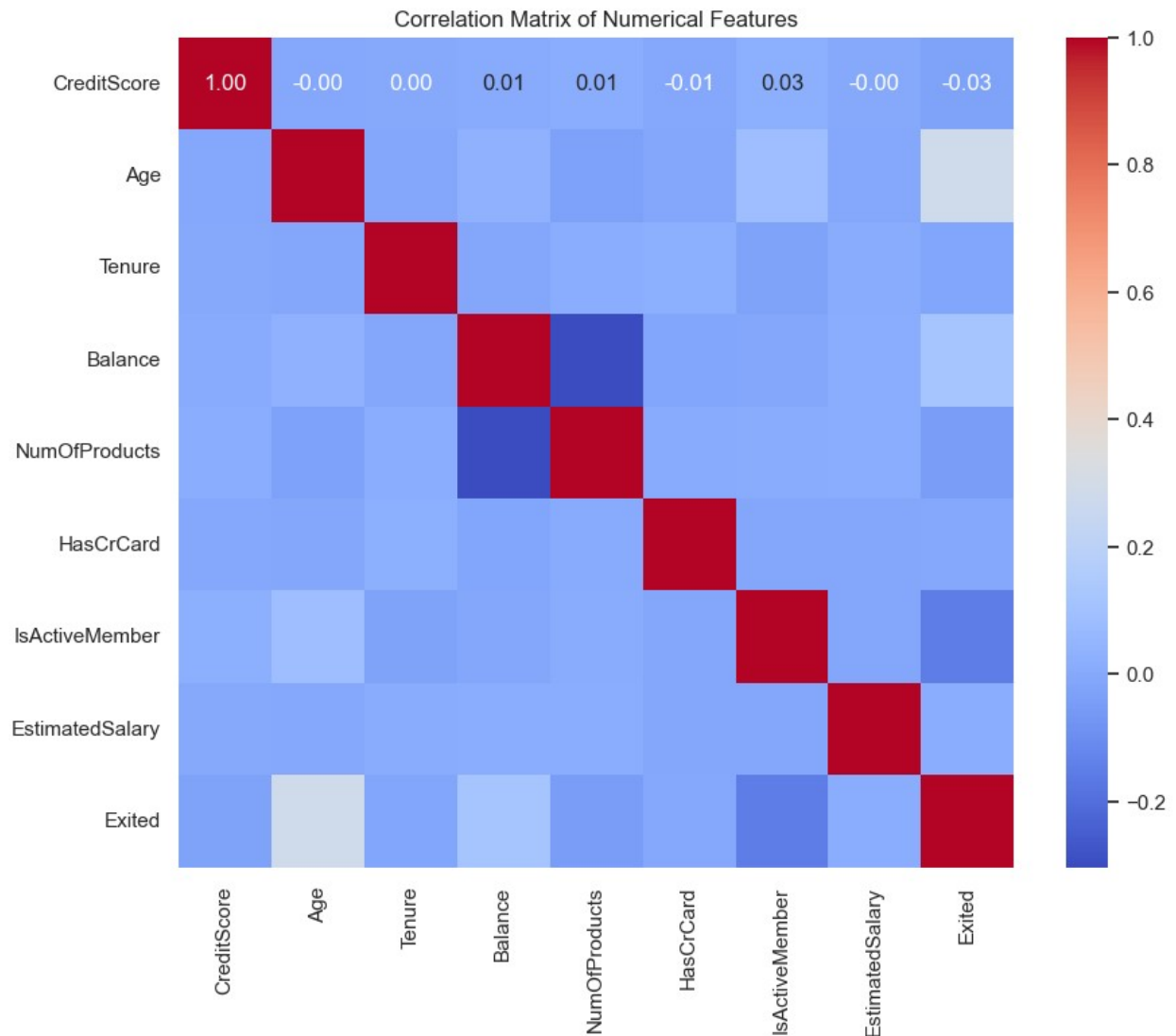
```
with pd.option_context('mode.use_inf_as_na', True):
D:\Users\HP\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119:
```

FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

```
with pd.option_context('mode.use_inf_as_na', True):
```



```
plt.figure(figsize=(10, 8))
numeric_ab = ab.drop(columns=['RowNumber', 'CustomerId', 'Surname',
                              'Geography', 'Gender'])
sns.heatmap(numeric_ab.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```

```
print(ab['Exited'].value_counts(normalize=True))
print(ab.groupby('Geography')['Exited'].mean())
print(ab.groupby('Gender')['Exited'].mean())
```

```
Exited
0    0.7963
1    0.2037
Name: proportion, dtype: float64
Geography
France    0.161548
Germany   0.324432
Spain     0.166734
Name: Exited, dtype: float64
Gender
Female    0.250715
```

Male 0.164559
Name: Exited, dtype: float64

Predictive Modeling Strategy

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler

X = ab.drop(columns=['RowNumber', 'CustomerId', 'Surname', 'Exited'])
y = ab['Exited']

X = pd.get_dummies(X, columns=['Geography', 'Gender'],
drop_first=True)

cols_to_scale = ['CreditScore', 'Age', 'Tenure', 'Balance',
'NumOfProducts', 'EstimatedSalary']

scaler = StandardScaler()
X[cols_to_scale] = scaler.fit_transform(X[cols_to_scale])

display(X.head())
print("Shape of X:", X.shape)
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard
0	-0.326221	0.293517	-1.041760	-1.225848	-0.911583	1
1	-0.440036	0.198164	-1.387538	0.117350	-0.911583	0
2	-1.536794	0.293517	1.032908	1.333053	2.527057	1
3	0.501521	0.007457	-1.387538	-1.225848	0.807737	0
4	2.063884	0.388871	-1.041760	0.785728	-0.911583	1

	IsActiveMember	EstimatedSalary	Geography_Germany	Geography_Spain
0	1	0.021886	False	False
1	1	0.216534	False	True
2	0	0.240687	False	False
3	0	-0.108918	False	False
4	1	-0.365276	False	True

```
Gender_Male
0      False
1      False
2      False
3      False
4      False
```

```
Shape of X: (10000, 11)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
```

```
rf_model = RandomForestClassifier(n_estimators=100,
class_weight='balanced', random_state=42)
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.87	0.97	0.92	1593
1	0.78	0.44	0.56	407
accuracy			0.86	2000
macro avg	0.82	0.70	0.74	2000
weighted avg	0.85	0.86	0.85	2000